

User Stories - ระบบนัดหมายคลินิกสุขภาพฯ (6 Features)

1. ระบบ Authentication & Profile (รับผิดชอบโดยคนที่ 1)

ID	User Story	Acceptance Criteria
US-A1	ในฐานะนักศึกษา/บุคลากร ฉันต้องการ login ก่อนเข้าใช้งานระบบ เพื่อป้องกันผู้อื่นสวมรอย	มีหน้า Login/Register เชื่อมต่อระบบ Auth ของ Supabase
US-A2	ในฐานะผู้ในระบบ ฉันต้องการดูข้อมูลส่วนตัว (ชื่อ, รหัสนักศึกษา, คณะ) เพื่อตรวจสอบความถูกต้อง	หน้า Profile ดึงข้อมูลผู้ใช้จาก DB จริงมาแสดงผล
US-A3	ในฐานะผู้ในระบบ ฉันต้องการแก้ไขเบอร์โทรศัพท์และข้อมูลติดต่อฉุกเฉินได้ เพื่อให้คลินิกติดต่อได้	ฟอร์ม update ข้อมูล พร้อม validation ตรวจสอบรูปแบบเบอร์โทรศัพท์
US-A4	ในฐานะระบบ ฉันต้องการป้องกันไม่ให้ผู้ที่ยังไม่ login เข้าถึงหน้าจองคิว เพื่อความปลอดภัย	มีระบบตรวจสอบ (Protected Route) หากยังไม่ login ให้ Redirect ไปหน้า Login

2. ระบบจองคิวนัดหมาย Appointment (รับผิดชอบโดยคนที่ 2)

ID	User Story	Acceptance Criteria
US-B1	ในฐานะนักศึกษา ฉันต้องการเห็นวันและเวลาที่คลินิกว่าง เพื่อเลือกจองคิว	หน้า /booking แสดงปฏิทินหรือ Slot เวลาที่ดึงจาก DB จริง (ช่องเวลาที่เต็มแล้ว)
US-B2	ในฐานะนักศึกษา ฉันต้องการกดจองคิวล่วงหน้า เพื่อเข้าพบแพทย์โดยไม่ต้องรอ	ฟอร์ม insert ข้อมูลการจองคิว พร้อมแสดงหน้า Success เมื่อจองสำเร็จ
US-B3	ในฐานะเจ้าหน้าที่ ฉันต้องการดูตารางนัดหมายทั้งหมดของวัน เพื่อเตรียมตัว	หน้า Dashboard ของ Admin แสดงตารางคิวผู้ป่วยเรียงตามเวลา

US-B4	ในฐานะนักศึกษา ฉันต้องการยกเลิกคิวนัดของตัวเอง เพื่อคืนคิวให้คนอื่น	ปุ่ม delete คิวของตนเอง พร้อม Confirm dialog ก่อนลบ
-------	---	---

3. ระบบบันทึกประวัติการรักษา Medical Record (รับผิดชอบโดยคนที่ 3)

ID	User Story	Acceptance Criteria
US-C1	ในฐานะแพทย์/เจ้าหน้าที่ ฉันต้องการบันทึกอาการป่วยและผลวินิจฉัย เพื่อเก็บประวัติ	ฟอร์ม insert ข้อมูลประวัติการรักษา (Text area) ผูกกับ ID ของผู้ป่วย
US-C2	ในฐานะแพทย์/เจ้าหน้าที่ ฉันต้องการแก้ไขผลการตรวจเพิ่มเติมได้	ฟอร์ม update ประวัติเดิม พร้อมบันทึก Timestamp ล่าสุดที่แก้ไข
US-C3	ในฐานะนักศึกษา ฉันต้องการดูประวัติการรักษาของตนเอง เพื่อทราบปัญหาสุขภาพย้อนหลัง	หน้า History แสดง Card ประวัติย้อนหลัง ดึงข้อมูลจาก DB
US-C4	ในฐานะระบบ ฉันต้องการป้องกันไม่ให้นักศึกษาดูประวัติคนอื่น เพื่อรักษาความลับผู้ป่วย	Policy (RLS ใน Supabase) จำกัดสิทธิ์ให้อ่านได้เฉพาะ row ที่ตรงกับ ID ตนเอง

4. ระบบสั่งจ่ายยา Prescription (รับผิดชอบโดยคนที่ 4)

ID	User Story	Acceptance Criteria
US-D1	ในฐานะแพทย์ ฉันต้องการเพิ่มรายการยาลงในใบสั่งยาผู้ป่วย เพื่อระบุยาที่ต้องจ่าย	ฟอร์ม insert ที่มี Dropdown ดึงรายชื่อยาจากตารางคลังยามาให้เลือก
US-D2	ในฐานะแพทย์ ฉันต้องการระบุจำนวนยาและวิธีกินยาให้ชัดเจน เพื่อให้ผู้ป่วยกินถูกต้อง	ฟอร์มกรอกจำนวน (Number) และ Dropdown วิธีกินยา (เช่น ก่อนอาหาร, หลังอาหาร)
US-D3	ในฐานะแพทย์ ฉันต้องการลบรายการยาที่ใส่ผิดออกจากใบสั่ง	ปุ่ม delete ท้ายรายการยา พร้อม Confirm dialog

	ยา เพื่อป้องกันการจ่ายยาผิด	
US-D4	ในฐานะนักศึกษา ฉันต้องการดูรายการยาที่ฉันได้รับ เพื่อทราบสรรพคุณและวิธีกิน	หน้า /prescription/[id] แสดงตารางรายการยาที่ได้รับในแต่ละครั้ง

5. ระบบแจ้งเตือนการกินยา Medication Reminder (รับผิดชอบโดยคนที่ 5)

ID	User Story	Acceptance Criteria
US-E1	ในฐานะระบบ ฉันต้องการสร้างรอบแจ้งเตือนตามใบสั่งยาอัตโนมัติ เพื่อประหยัดเวลาดังค่า	มี Logic (API/Trigger) อ่านวิธีกินยาจากใบสั่งยา แล้ว Insert รอบเวลาลงตารางแจ้งเตือน
US-E2	ในฐานะนักศึกษา ฉันต้องการเห็นการแจ้งเตือนเมื่อถึงเวลากินยา เพื่อให้กินยาตรงเวลา	หน้า UI มี Badge แจ้งเตือน (Notification) หรือ Alert state ชัดเจนเมื่อถึงเวลา
US-E3	ในฐานะนักศึกษา ฉันต้องการกดยืนยันว่ากินยาแล้ว เพื่อบันทึกพฤติกรรม	ปุ่ม Update สถานะ (เช่น Checkbox) เปลี่ยนสถานะจาก "รอกิน" เป็น "กินแล้ว"
US-E4	ในฐานะนักศึกษา ฉันต้องการลบการแจ้งเตือนของยาที่กินครบโดสแล้ว เพื่อไม่ให้รบกวน	ปุ่ม delete รายการแจ้งเตือน พร้อม Confirm dialog

6. ระบบจัดการคลังยา Drug Inventory (รับผิดชอบโดยคนที่ 6)

ID	User Story	Acceptance Criteria
US-F1	ในฐานะเจ้าหน้าที่ ฉันต้องการเพิ่มรายชื้อและสต็อกยาใหม่เข้าคลัง เพื่อบันทึกข้อมูล	ฟอร์ม insert ข้อมูลยา (ชื่อ, รูปภาพ, จำนวน, วันหมดอายุ)
US-F2	ในฐานะเจ้าหน้าที่ ฉันต้องการดูจำนวนสต็อกยาคงเหลือทั้งหมด เพื่อสั่งซื้อเพิ่ม	หน้า /inventory แสดงตารางรายการยา ดึงข้อมูลจาก DB พร้อมระบบค้นหา (Search)

US-F3	ในฐานะระบบ จำต้องการตัดสต็อกยาอัตโนมัติเมื่อมีการสั่งจ่ายยา เพื่อให้จำนวนถูกต้องเสมอ	อัปเดตจำนวนยา (ลดลง) อัตโนมัติเมื่อฟอร์มจ่ายยา (US-D1/D2) ทำงานสำเร็จ
US-F4	ในฐานะเจ้าหน้าที่ จำต้องการลบยาที่หมดอายุออกจากระบบ เพื่อความปลอดภัย	ปุ่ม delete รายการยา พร้อม Confirm dialog และแสดง Error State หากพยายามลบยาที่มีการเชื่อมโยงกับใบสั่งยา

การวิเคราะห์และออกแบบระบบ "นัดหมายคลินิก สุขภาพภายในมหาวิทยาลัยและแจ้งเตือนการกินยา"

เอกสารสำหรับ: Product Backlog / Requirements Specification **Tech Stack:** Web Application
(Deploy บน Vercel) + Supabase (Backend & Database) **ทีมพัฒนา:** 6 คน (รับผิดชอบคนละ 1 ฟีเจอร์)
เอกสารนี้เป็นฐานสำหรับ: Use Case Diagram และ ER Diagram (ขั้นตอนถัดไป)

สารบัญ

- ภาพรวมระบบ
- ภาพรวมตารางฐานข้อมูล (สำหรับเตรียม ER Diagram)
 - User Roles
 - User Journey
 - Features Specification (6 ฟีเจอร์)
 - User Stories (Functional & Non-Functional)
 - Acceptance Criteria (แยกตามฟีเจอร์)
- สรุปขั้นตอนถัดไป

ภาพรวมระบบ

ระบบนี้รวม 2 บริการหลักไว้ในแพลตฟอร์มเดียว คือ (1) ระบบจองคิวนัดหมายกับคลินิกสุขภาพของมหาวิทยาลัย และ (2) ระบบแจ้งเตือนการกินยาส่วนบุคคล สำหรับให้นักศึกษาติดตามการกินยาตามที่แพทย์สั่งอย่างต่อเนื่อง ทั้งสองบริการเชื่อมโยงกันผ่านผู้ใช้งานเดียวกัน และแชร์โครงสร้าง Authentication/Authorization เดียวกันผ่าน Supabase Auth + Row Level Security (RLS)

ภาพรวมตารางฐานข้อมูล (สำหรับเตรียม ER Diagram)

โจทย์กำหนดขั้นต่ำ 6 ตาราง แต่เนื่องจากระบบนี้รวม 2 บริการย่อยเข้าด้วยกัน การออกแบบด้านล่างจึงใช้ 9 ตาราง เพื่อให้ข้อมูลเป็นระเบียบ (Normalized) และรองรับการแบ่งงานที่ชัดเจนระหว่างสมาชิก 6 คน

ตาราง	วัตถุประสงค์	ฟิลด์สำคัญ (โดยสังเขป)
profiles	ข้อมูลผู้ใช้ทุกคน (ต่อยอดจาก auth.users ของ Supabase)	id (FK→auth.users), student_id, full_name, phone, role, avatar_url
departments	แผนก/บริการของคลินิก	id, name, description
doctors	ข้อมูลแพทย์/บุคลากรทางการแพทย์	id, full_name, specialty, department_id (FK)
appointment_slots	ช่วงเวลาว่างที่เปิดให้จอง	id, doctor_id (FK), slot_date, start_time, end_time, max_capacity, booked_count, status
appointments	รายการนัดหมายที่ถูกจองจริง	id, user_id (FK), slot_id (FK), reason, status
medications	ฐานข้อมูลกลางของยา (Master Data)	id, name, generic_name, dosage_form, standard_dosage, warnings
medication_reminders	รายการยาส่วนตัวที่ผู้ใช้ตั้งแจ้งเตือน	id, user_id (FK), medication_id (FK), reminder_times, start_date, end_date, status
medication_logs	ประวัติการกินยาจริง	id, reminder_id (FK), scheduled_datetime, actual_datetime, status
notifications	ศูนย์รวมการแจ้งเตือนทุกประเภท	id, user_id (FK), type, title, message, is_read

ความสัมพันธ์หลัก (สำหรับ ER Diagram): profiles (1:M) appointments, profiles (1:M) medication_reminders, departments (1:M) doctors, doctors (1:M) appointment_slots, appointment_slots (1:M) appointments, medications (1:M) medication_reminders, medication_reminders (1:M) medication_logs, profiles (1:M) notifications

1. User Roles

Role 1: นักศึกษา (Student / ผู้ใช้ทั่วไป)

ผู้ใช้ทั่วไปที่ลงทะเบียนด้วยอีเมล/รหัสนักศึกษา ใช้งานเพื่อจองคิวนัดหมายและจัดการรายการแจ้งเตือนยาของตนเอง

Role 2: เจ้าหน้าที่คลินิก (Staff / Admin)

ผู้ดูแลระบบฝั่งคลินิก จัดการตารางแพทย์ อนุมัติ/ปฏิเสธนัดหมาย จัดการฐานข้อมูลกลาง และดูภาพรวมสถิติการใช้งาน

ตารางสิทธิ์การเข้าถึงข้อมูล (Permission Matrix)

ตาราง	นักศึกษา (Student)	เจ้าหน้าที่ (Staff)
profiles	C (สมัคร) / R, U, D เฉพาะแถวตนเอง	R ทุกแถว
departments	R	C, R, U, D
doctors	R	C, R, U, D
appointment_slots	R	C, R, U, D
appointments	C, R, U, D เฉพาะแถวตนเอง (U/D จำกัดเมื่อ status='pending')	R ทุกแถว, U (เปลี่ยนสถานะ)
medications	R	C, R, U, D
medication_reminders	C, R, U, D เฉพาะแถวตนเอง	ไม่มีสิทธิ์เข้าถึง (ข้อมูลส่วนตัวด้านสุขภาพ)
medication_logs	C, R เฉพาะแถวตนเอง	ไม่มีสิทธิ์เข้าถึง

notifications	R, U (mark read), D เฉพาะแอดตนเอง	C (broadcast), R, U, D เฉพาะแอดตนเอง
---------------	-----------------------------------	--------------------------------------

2. User Journey

Journey A: นักศึกษา (Student)

1. เข้าเว็บไซต์ → เห็นหน้า Landing พร้อมปุ่ม Login / สมัครสมาชิก
2. สมัครสมาชิกด้วยอีเมลมหาวิทยาลัย รหัสนักศึกษา และรหัสผ่าน → ระบบสร้างบัญชีใน Supabase Auth และสร้างแถวใน `profiles` (role='student')
3. Login → เข้าสู่หน้า Dashboard ของนักศึกษา
4. **เส้นทางนัดหมาย:** เลือกเมนู "นัดหมายคลินิก" → เลือกแผนกที่ต้องการ → ดูตารางเวลาว่างของแพทย์ (ดึงจาก `appointment_slots`) → เลือกวัน-เวลา → กรอกอาการ/เหตุผล → กดยืนยันจอง → ระบบบันทึกลง `appointments` (status='pending') → ได้รับ Notification "รอการยืนยัน"
5. ดู "นัดหมายของฉัน" → ติดตามสถานะ → เลื่อนนัด/ยกเลิกนัดได้ (ขณะยัง pending) → เมื่อเจ้าหน้าที่ ยืนยัน จะได้รับ Notification แจ้งผล
6. **เส้นทางแจ้งเตือนยา:** เลือกเมนู "ยาของฉัน" → กด "เพิ่มรายการยา" → ค้นหาจากฐานข้อมูลกลาง → กำหนดขนาดยา ความถี่ และเวลาแจ้งเตือน → บันทึกลง `medication_reminders`
7. เมื่อถึงเวลา ระบบแจ้งเตือนในแอป → กด "กินแล้ว"/"ข้าม" → บันทึกลง `medication_logs`
8. ดูประวัติการกินยา/อัตราความสม่ำเสมอย้อนหลัง
9. ดูศูนย์การแจ้งเตือนรวม → อ่าน/ลบแจ้งเตือน → แก้ไขโปรไฟล์ หรือ Logout

Journey B: เจ้าหน้าที่คลินิก (Staff)

1. Login ด้วยบัญชี Staff → เข้าสู่หน้า Admin Dashboard (เห็นสถิติภาพรวมวันนี้)
2. จัดการแผนก/แพทย์: เพิ่ม/แก้ไข/ลบข้อมูลแผนกและแพทย์
3. สร้างตารางเวลาว่างของแพทย์แต่ละคน (วันที่ เวลา จำนวนคิว) → เปิดให้นักศึกษาจอง
4. ดูรายการนัดหมายที่รอดำเนินการ → ยืนยัน/ปฏิเสธ (พร้อมเหตุผล) → เมื่อถึงวันนัดและตรวจเสร็จ กด "เสร็จสิ้น"
5. จัดการฐานข้อมูลยากลาง: เพิ่ม/แก้ไข/ลบข้อมูลยา
6. ดูแดชบอร์ดสรุปสถิติ (จำนวนนัดหมาย แผนกยอดนิยม อัตรายกเลิก)
7. ส่งประกาศ/แจ้งเตือนไปยังผู้ใช้ทั้งหมดหรือเฉพาะกลุ่ม
8. Logout

3. Features Specification (6 ฟีเจอร์)

#	ฟีเจอร์	ตารางหลักที่รับผิดชอบ	Role ที่เกี่ยวข้อง
---	---------	-----------------------	--------------------

1	ระบบสมาชิกและโปรไฟล์ผู้ใช้ (Authentication & Profile)	profiles	Student, Staff
2	ระบบจัดการแผนกและตารางเวลาแพทย์ (Department & Schedule)	departments, doctors, appointment_slots	Staff (C/U/D), Student (R)
3	ระบบนัดหมายคลินิก (Appointment Booking)	appointments	Student (C/R/U/D own), Staff (R/U)
4	ระบบจัดการฐานข้อมูลยา (Medication Master Data)	medications	Staff (C/U/D), Student (R)
5	ระบบแจ้งเตือนการกินยาส่วนบุคคล (Medication Reminder)	medication_reminders, medication_logs	Student (C/R/U/D own)
6	ศูนย์แจ้งเตือนและแดชบอร์ด (Notification Center & Dashboard)	notifications	Student (R/U/D own), Staff (C/R)

ฟีเจอร์ที่ 1: ระบบสมาชิกและโปรไฟล์ผู้ใช้

สมัครสมาชิก เข้าสู่ระบบ ออกจากระบบ และจัดการข้อมูลส่วนตัวของผู้ใช้ทั้ง 2 Role ผ่าน Supabase Auth
CRUD: Create (สมัคร/auto-create profile) · Read (ดูโปรไฟล์) · Update (แก้ไขข้อมูล) · Delete (ลบ/ปิดใช้งานบัญชี)

ฟีเจอร์ที่ 2: ระบบจัดการแผนกและตารางเวลาแพทย์

เจ้าหน้าที่จัดการข้อมูลแผนก แพทย์ และช่วงเวลาที่เปิดให้จอง นักศึกษาค้นหาและดูช่วงเวลาว่าง **CRUD:** Create (เพิ่มแผนก/แพทย์/Slot) · Read (ดูรายการ/ค้นหา) · Update (แก้ไขข้อมูล/เลื่อน Slot) · Delete (ลบแผนก/แพทย์/Slot ที่ยังไม่ถูกจอง)

ฟีเจอร์ที่ 3: ระบบนัดหมายคลินิก

แกนกลางของระบบ: นักศึกษาจองคิว ดูสถานะ เลื่อน/ยกเลิกนัด เจ้าหน้าที่อนุมัติ/ปฏิเสธ/ปิดงาน **CRUD:** Create (จองนัด) · Read (ดูนัดหมาย) · Update (เลื่อนนัด/เปลี่ยนสถานะ) · Delete (ยกเลิกนัด)

ฟีเจอร์ที่ 4: ระบบจัดการฐานข้อมูลยา

เจ้าหน้าที่ดูแลฐานข้อมูลยากกลางที่นักศึกษาใช้อ้างอิงตอนตั้งการแจ้งเตือน **CRUD:** Create (เพิ่มยาใหม่) · Read

(ค้นหา/ดูรายละเอียด) · Update (แก้ไขข้อมูลยา) · Delete (ลบ/ปิดใช้งานยา)

ฟีเจอร์ที่ 5: ระบบแจ้งเตือนการกินยาส่วนบุคคล

นักศึกษาสร้างรายการยาที่ตนเองต้องกิน กำหนดตารางแจ้งเตือน และบันทึกผลการกินยาจริง **CRUD:** Create (เพิ่มรายการยา/บันทึก Log) · Read (ดูรายการ/ประวัติ) · Update (แก้ไข/หยุดชั่วคราว) · Delete (ลบรายการ)

ฟีเจอร์ที่ 6: ศูนย์แจ้งเตือนและแดชบอร์ด

รวมการแจ้งเตือนทุกประเภทไว้ที่เดียว พร้อมแดชบอร์ดสรุปสถิติสำหรับเจ้าหน้าที่ **CRUD:** Create (ระบบ/Staff สร้างแจ้งเตือน) · Read (ดูรายการ/สถิติ) · Update (mark read) · Delete (ลบแจ้งเตือน)

4. User Stories (Functional & Non-Functional)

ฟีเจอร์ที่ 1: ระบบสมาชิกและโปรไฟล์ผู้ใช้

- **1.1 (Functional):** ในฐานะนักศึกษาใหม่, ฉันต้องการสมัครสมาชิกด้วยอีเมลและรหัสนักศึกษา, เพื่อที่จะสามารถเข้าใช้งานระบบนัดหมายและแจ้งเตือนยาได้
- **1.2 (Functional):** ในฐานะผู้ใช้ที่ลงทะเบียนแล้ว, ฉันต้องการเข้าสู่ระบบด้วยอีเมลและรหัสผ่าน, เพื่อที่จะเข้าถึงข้อมูลส่วนตัวและฟีเจอร์ต่างๆ ได้
- **1.3 (Functional):** ในฐานะผู้ใช้ที่เข้าสู่ระบบแล้ว, ฉันต้องการดูและแก้ไขข้อมูลโปรไฟล์ของตนเอง, เพื่อที่จะให้ข้อมูลติดต่อเป็นปัจจุบันอยู่เสมอ
- **1.4 (Functional):** ในฐานะผู้ใช้, ฉันต้องการออกจากระบบและลบบัญชีของตนเองได้, เพื่อที่จะควบคุมความเป็นส่วนตัวของข้อมูลตนเอง
- **1.5 (Non-Functional):** ในฐานะผู้ใช้, ฉันต้องการให้ระบบป้องกันไม่ให้อื่นเข้าถึงหรือแก้ไขข้อมูลโปรไฟล์ของฉันได้, เพื่อที่จะมั่นใจในความปลอดภัยของข้อมูลส่วนตัว
- **1.6 (Non-Functional):** ในฐานะผู้ใช้, ฉันต้องการให้ระบบตอบสนองต่อการเข้าสู่ระบบภายในเวลาไม่เกิน 2 วินาที, เพื่อที่จะไม่เสียเวลาในการเข้าใช้งาน

ฟีเจอร์ที่ 2: ระบบจัดการแผนกและตารางเวลาแพทย์

- **2.1 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการเพิ่ม แก้ไข และลบข้อมูลแผนกและแพทย์, เพื่อที่จะจัดการข้อมูลบริการของคลินิกให้เป็นปัจจุบัน
- **2.2 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการสร้างตารางเวลาว่างของแพทย์แต่ละคน, เพื่อที่จะเปิดให้นักศึกษาจองคิวได้
- **2.3 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการแก้ไขหรือลบช่วงเวลาที่ยังไม่มีผู้จอง, เพื่อที่จะปรับตารางให้สอดคล้องกับความพร้อมของแพทย์จริง
- **2.4 (Functional):** ในฐานะนักศึกษา, ฉันต้องการค้นหาและดูตารางเวลาว่างของแพทย์ตามแผนกที่ต้องการ, เพื่อที่จะเลือกเวลานัดหมายที่สะดวก
- **2.5 (Non-Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการให้ระบบป้องกันไม่ให้ลบช่วงเวลาที่มีการจอง

แล้ว, เพื่อที่จะป้องกันข้อมูลนัดหมายของนักศึกษาเสียหาย

ฟีเจอร์ที่ 3: ระบบนัดหมายคลินิก

- **3.1 (Functional):** ในฐานะนักศึกษา, ฉันต้องการจองคิวนัดหมายกับแพทย์ตามวันเวลาที่ว่าง, เพื่อที่จะเข้ารับบริการตรวจสุขภาพตามเวลาที่สะดวก
- **3.2 (Functional):** ในฐานะนักศึกษา, ฉันต้องการดูรายการนัดหมายของตนเองทั้งหมดพร้อมสถานะ, เพื่อที่จะติดตามว่านัดหมายได้รับการยืนยันหรือไม่
- **3.3 (Functional):** ในฐานะนักศึกษา, ฉันต้องการยกเลิกหรือเลื่อนนัดหมายที่ยังไม่ถึงกำหนด, เพื่อที่จะบริหารเวลาของตนเองเมื่อมีเหตุขัดข้อง
- **3.4 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการยืนยันหรือปฏิเสธคำขอนัดหมายที่เข้ามา, เพื่อที่จะควบคุมจำนวนผู้ป่วยให้เหมาะสมกับแพทย์ที่มี
- **3.5 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการปรับสถานะนัดหมายเป็น "เสร็จสิ้น" หลังตรวจเสร็จ, เพื่อที่จะปิดงานและเก็บสถิติการให้บริการ
- **3.6 (Non-Functional):** ในฐานะนักศึกษา, ฉันต้องการให้ระบบป้องกันการจองคิวซ้อนทับกันในช่วงเวลาเดียวกัน, เพื่อที่จะไม่เกิดความสับสนเรื่องเวลานัดหมาย

ฟีเจอร์ที่ 4: ระบบจัดการฐานข้อมูลยา

- **4.1 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการเพิ่มข้อมูลยาใหม่เข้าสู่ฐานข้อมูลกลาง, เพื่อที่จะให้นักศึกษามีข้อมูลอ้างอิงที่ถูกต้อง
- **4.2 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการแก้ไขหรือลบข้อมูลยาที่ไม่ถูกต้องหรือเลิกใช้แล้ว, เพื่อที่จะรักษาความถูกต้องของฐานข้อมูลยา
- **4.3 (Functional):** ในฐานะนักศึกษา, ฉันต้องการค้นหาข้อมูลยาจากฐานข้อมูลกลาง, เพื่อที่จะเลือกยาที่ถูกต้องเมื่อสร้างรายการแจ้งเตือน
- **4.4 (Non-Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการให้เฉพาะผู้มีสิทธิ์ Admin เท่านั้นที่แก้ไขฐานข้อมูลยาได้, เพื่อที่จะป้องกันข้อมูลยาผิดพลาดจากผู้ใช้ทั่วไป

ฟีเจอร์ที่ 5: ระบบแจ้งเตือนการกินยาส่วนบุคคล

- **5.1 (Functional):** ในฐานะนักศึกษา, ฉันต้องการเพิ่มรายการยาที่ตนเองต้องกินพร้อมกำหนดเวลาแจ้งเตือน, เพื่อที่จะไม่ลืมกินยาตามที่แพทย์สั่ง
- **5.2 (Functional):** ในฐานะนักศึกษา, ฉันต้องการแก้ไขหรือหยุดรายการแจ้งเตือนยาที่ตั้งไว้, เพื่อที่จะปรับให้ตรงกับแผนการรักษาที่เปลี่ยนแปลง
- **5.3 (Functional):** ในฐานะนักศึกษา, ฉันต้องการกดยืนยันว่า "กินยาแล้ว" หรือ "ข้ามมื้อนี้" เมื่อได้รับแจ้งเตือน, เพื่อที่จะบันทึกประวัติการกินยาของตนเอง
- **5.4 (Functional):** ในฐานะนักศึกษา, ฉันต้องการดูสรุปประวัติการกินยาย้อนหลัง, เพื่อที่จะประเมินความสม่ำเสมอในการกินยาของตนเอง
- **5.5 (Non-Functional):** ในฐานะนักศึกษา, ฉันต้องการให้ข้อมูลยาส่วนตัวของฉันมองเห็นได้เฉพาะตัวฉันเองเท่านั้น, เพื่อที่จะรักษาความเป็นส่วนตัวด้านสุขภาพ

ฟีเจอร์ที่ 6: ศูนย์แจ้งเตือนและแดชบอร์ด

- **6.1 (Functional):** ในฐานะผู้ใช้, ฉันต้องการเห็นรายการแจ้งเตือนทั้งหมด (นัดหมาย/ยา) รวมอยู่ในที่เดียว, เพื่อที่จะไม่พลาดข้อมูลสำคัญ
- **6.2 (Functional):** ในฐานะผู้ใช้, ฉันต้องการทำเครื่องหมายแจ้งเตือนว่าอ่านแล้วหรือลบแจ้งเตือนที่ไม่ต้องการ, เพื่อที่จะจัดการกล่องแจ้งเตือนให้เป็นระเบียบ
- **6.3 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการดูแดชบอร์ดสรุปสถิติการนัดหมายและการทำงานของระบบ, เพื่อที่จะวางแผนการให้บริการคลินิกได้อย่างมีประสิทธิภาพ
- **6.4 (Functional):** ในฐานะเจ้าหน้าที่, ฉันต้องการส่งประกาศแจ้งเตือนไปยังผู้ใช้ทั้งหมดหรือเฉพาะกลุ่ม, เพื่อที่จะสื่อสารข้อมูลสำคัญของคลินิกได้อย่างรวดเร็ว
- **6.5 (Non-Functional):** ในฐานะผู้ใช้, ฉันต้องการให้ระบบส่งการแจ้งเตือนได้ตรงเวลาโดยคลาดเคลื่อนไม่เกิน 1 นาที, เพื่อที่จะมั่นใจว่าจะไม่พลาดนัดหมายหรือเวลากินยา

5. Acceptance Criteria (แยกตามฟีเจอร์)

ตาราง AC — ฟีเจอร์ที่ 1: ระบบสมาชิกและโปรไฟล์ผู้ใช้

ID	User Story	Acceptance Criteria
1.1	สมัครสมาชิกด้วยอีเมลและรหัสนักศึกษา	1. มี Form สมัครสมาชิก (อีเมล, รหัสผ่าน, ยืนยันรหัสผ่าน, ชื่อ-นามสกุล, รหัสนักศึกษา, เบอร์โทร) พร้อม Validation ครบทุกฟิลด์ 2. กดสมัคร → เรียก Supabase Auth signUp() และ INSERT แถวใหม่ในตาราง profiles (role='student' โดย default) แบบ atomic 3. แสดง Loading Spinner และ ปิดปุ่ม Submit ระหว่างส่งข้อมูล (ป้องกันกดซ้ำ) 4. หากอีเมลซ้ำหรือเกิด Error แสดงข้อความ Error ใต้ Form โดยไม่ redirect ออก 5. สมัครสำเร็จ → redirect ไปหน้า Login/Dashboard พร้อม Toast ยืนยัน 6. RLS Policy บน profiles : INSERT อนุญาตเฉพาะเมื่อ auth.uid() = id ที่กำลังสร้าง
1.2	เข้าสู่ระบบด้วยอีเมลและรหัสผ่าน	1. มี Form Login (อีเมล, รหัสผ่าน) พร้อมลิงก์ "ลืมรหัสผ่าน" 2. เรียก Supabase Auth signInWithPassword() พร้อมแสดง Loading State 3. สำเร็จ → ดึง role จาก profiles แล้ว redirect ตาม role (student/staff) 4. ไม่สำเร็จ → แสดง Error message ทั่วไป ("อีเมลหรือรหัสผ่านไม่ถูกต้อง") โดยไม่ระบุว่ามีฟิลด์ใดผิด 5. หน้าที่ต้อง Login (Protected Route) ต้อง redirect ผู้ใช้ที่ยังไม่ Login กลับหน้า Login อัตโนมัติ

1.3	ดูและแก้ไขโปรไฟล์ของตนเอง	1. ดึงข้อมูลจริงจาก <code>profiles</code> ด้วย <code>SELECT ... WHERE id = auth.uid()</code> มาแสดงในฟอร์ม 2. แสดง Loading Skeleton ขณะโหลด และ Error State พร้อมปุ่ม "ลองใหม่" หากล้มเหลว 3. ฟอร์มแก้ไขได้เฉพาะ <code>full_name</code> , <code>phone</code> , <code>avatar_url</code> (ห้ามแก้ไข <code>email/role</code>) 4. กดบันทึก → เรียก UPDATE พร้อม Toast แจ้งผลสำเร็จ/ล้มเหลว 5. RLS Policy: UPDATE อนุญาตเฉพาะ <code>auth.uid() = id</code> ของแอดมิน
1.4	ออกจากระบบและลบบัญชี	1. ปุ่ม "ออกจากระบบ" เรียก <code>signOut()</code> แล้ว redirect หน้า Login ทันที 2. เมนู "ลบบัญชี" ต้องมี Confirm Dialog เตือนว่าไม่สามารถย้อนกลับได้ 3. ยืนยันแล้ว → ลบ/ปิดใช้งานแอดใน <code>profiles</code> แล้ว sign out ทันที 4. แสดง Loading State ระหว่างดำเนินการ และ Error State หากล้มเหลว
1.5	ป้องกันการเข้าถึงข้อมูลโปรไฟล์ผู้อื่น	1. รหัสผ่านจัดการผ่าน Supabase Auth เท่านั้น ไม่เก็บ plain-text ในตารางใดๆ 2. เปิดใช้งาน RLS (Enable RLS) บนตาราง <code>profiles</code> 3. Policy SELECT: นักศึกษาเห็นเฉพาะแอดตนเอง (<code>auth.uid() = id</code>); <code>role='staff'</code> เห็นได้ทุกแอด
1.6	เข้าสู่ระบบได้ภายใน 2 วินาที	1. วัด Time-to-Interactive ของหน้า Login (จากกด Submit ถึง Redirect) ต้องไม่เกิน 2 วินาทีบน Production (Vercel) 2. แสดง Loading Indicator ทันทีที่กด Submit เพื่อ Feedback แก่ผู้ใช้

ตาราง AC — ฟังก์ชันที่ 2: ระบบจัดการแผนกและตารางเวลาแพทย์

ID	User Story	Acceptance Criteria
2.1	Admin เพิ่ม/แก้ไข/ลบแผนกและแพทย์	1. Form เพิ่มแผนก (ชื่อ, คำอธิบาย) และ Form เพิ่มแพทย์ (ชื่อ, ความเชี่ยวชาญ, เลือกแผนกจาก Dropdown ที่ดึงจาก <code>departments</code> จริง, อัปโหลดรูป) 2. ตารางรายการดึงข้อมูลจริงจาก DB แสดงพร้อมปุ่ม Edit/Delete ทุกแถว 3. ปุ่ม Delete ต้องเปิด Confirm Dialog ก่อนเรียก DELETE จริง 4. Loading State ขณะโหลดตาราง, Error State พร้อมปุ่ม Retry หากล้มเหลว 5. RLS: SELECT เปิดให้ authenticated ทุกคน, INSERT/UPDATE/DELETE จำกัดเฉพาะ <code>role='staff'</code>

2.2	สร้างตาราง เวลาว่างของ แพทย์	1. Form สร้าง Slot: เลือกแพทย์ (Dropdown), วันที่ (Date Picker), เวลาเริ่ม-สิ้นสุด (Time Picker), จำนวนคิวสูงสุด พร้อม Validation ห้ามเวลาซ้อนทับของแพทย์คนเดียวกัน 2. บันทึก → INSERT ลง appointment_slots (status='available', booked_count=0) พร้อม Toast ยืนยัน 3. ตารางแสดงรายการ Slot พร้อม Filter ตามแพทย์/วันที่ ดึงข้อมูลจริงจาก DB
2.3	แก้ไข/ลบ ช่วงเวลาที่ยัง ไม่มีผู้จอง	1. ปุ่ม Edit เปิด Form Pre-filled จากข้อมูลเดิม; ปุ่ม Delete มี Confirm Dialog 2. หาก booked_count > 0 ต้อง disable ปุ่ม Delete/Edit พร้อมข้อความอธิบายเหตุผล 3. Loading/Error State ครบถ้วนตามมาตรฐาน
2.4	ค้นหาและดู ตารางเวลา ว่างของ แพทย์	1. หน้าค้นหาจะมี Filter (แผนก, วันที่, แพทย์) ดึง appointment_slots ที่ status='available' AND slot_date >= วันนี้ จาก DB จริง 2. Loading Skeleton ขณะค้นหา, ข้อความ "ไม่พบช่วงเวลาว่าง" หากผลลัพธ์ว่าง 3. RLS: SELECT เปิดให้ authenticated ทุกคนอ่านได้
2.5	ป้องกันการ ลบช่วงเวลา ที่มีการจอง แล้ว	1. Database Constraint/Trigger ตรวจสอบก่อน DELETE บน appointment_slots : หากมี appointments ที่อ้างอิงอยู่ (status ≠ cancelled) ให้ปฏิเสธการลบและคืน Error 2. Frontend ต้องจับ Error นี้และแสดงข้อความอธิบายให้ผู้ใช้งานทราบ

ตาราง AC — ฟังก์ชันที่ 3: ระบบนัดหมายคลินิก

ID	User Story	Acceptance Criteria
3.1	จองคิว นัดหมายกับ แพทย์	1. หลังเลือก Slot แสดง Form ยืนยัน (ชื่อแพทย์/แผนก/วันเวลา, ช่องกรอกอาการ) 2. กดยืนยัน → INSERT ลง appointments (status='pending') พร้อมอัปเดต booked_count ใน appointment_slots (+1) ภายใน Transaction เดียวกัน 3. Loading State ระหว่างบันทึก, ปิดปุ่มกันกดซ้ำ, Error State หาก Slot เต็มพอดีก่อนบันทึกสำเร็จ (Race Condition) 4. บันทึกสำเร็จ → สร้าง Notification อัตโนมัติ ("การจองยืนยัน") และ redirect ไปหน้า "นัดหมายของฉัน"

3.2	ดูรายการนัดหมายของตนเองพร้อมสถานะ	1. ดึงข้อมูลจริงจาก <code>appointments JOIN appointment_slots JOIN doctors</code> WHERE <code>user_id = auth.uid()</code> เรียงตามวันที่ 2. แสดงสถานะด้วย Badge สี (เหลือง=pending, เขียว=confirmed, แดง=rejected/cancelled, เทา=completed) 3. Loading Skeleton ขณะโหลด, Empty State ("คุณยังไม่มีนัดหมาย") หากไม่มีข้อมูล
3.3	ยกเลิก/เลื่อนนัดหมายที่ยังไม่ถึงกำหนด	1. ปุ่ม "เลื่อนนัด" ใช้ได้เฉพาะสถานะ pending (disable สำหรับสถานะอื่น) เปิด Form เลือก Slot ใหม่ 2. ปุ่ม "ยกเลิกนัด" แสดง Confirm Dialog ก่อนเรียก UPDATE <code>status='cancelled'</code> และลด <code>booked_count</code> ของ slot เดิม -1 3. RLS: UPDATE/DELETE โดยนักศึกษาอนุญาตเฉพาะ <code>auth.uid() = user_id AND status = 'pending'</code>
3.4	ยืนยัน/ปฏิเสธคำขอนัดหมาย	1. หน้า Admin ดึง <code>appointments</code> ที่ <code>status='pending'</code> จาก DB จริง พร้อม Filter ตามแผนก/วันที่ 2. ปุ่ม "ยืนยัน"/"ปฏิเสธ" เรียก UPDATE ทันที (ปฏิเสธต้องกรอกเหตุผลก่อนบันทึก) และสร้าง Notification แจ้งนักศึกษาเจ้าของนัดหมาย 3. RLS: UPDATE บน <code>appointments</code> อนุญาตเฉพาะ <code>role='staff'</code> ; Loading/Error State ตามมาตรฐาน
3.5	ปรับสถานะนัดหมายเป็น "เสร็จสิ้น"	1. ปุ่ม "เสร็จสิ้น" ปรากฏเฉพาะนัดหมาย <code>status=confirmed</code> ที่วันที่ตรงหรือผ่านมาแล้ว เรียก UPDATE <code>status='completed'</code> 2. Confirm Dialog ก่อนเปลี่ยนสถานะ เนื่องจากเป็น Action ที่ปิดงานถาวร
3.6	ป้องกันการจองคิวซ้อนทับกัน	1. Database-level Constraint/Trigger ป้องกัน <code>user_id</code> เดียวกันมี <code>appointments</code> สถานะ pending/confirmed ซ้อนช่วงเวลาเดียวกัน 2. Frontend แสดง Error message ชัดเจนหากระบบปฏิเสธการจองซ้อน

ตาราง AC — ฟังก์ชันที่ 4: ระบบจัดการฐานข้อมูลยา

ID	User Story	Acceptance Criteria
4.1	เพิ่มข้อมูลยาใหม่	1. Form เพิ่มยา: ชื่อยา, ชื่อสามัญ, รูปแบบยา (Dropdown), ขนาดมาตรฐาน, คำอธิบาย, คำเตือน (Textarea) พร้อม Validation required บนชื่อยา 2. บันทึก → INSERT ลง <code>medications</code> พร้อม Loading State และ Toast ยืนยัน, Error State หากล้มเหลว

4.2	แก้ไข/ลบข้อมูลยา	1. ตารางรายการยาดึงข้อมูลจริง พร้อมช่องค้นหาแบบ debounce 2. ปุ่ม Edit เปิด Form Pre-filled; ปุ่ม Delete มี Confirm Dialog ก่อนเรียก DELETE จริง 3. หากมี <code>medication_reminders</code> อ้างอิง <code>medication_id</code> นี้อยู่ แนะนำใช้ Soft Delete (<code>is_active=false</code>) แทนการลบถาวร พร้อมข้อความแจ้งเตือน
4.3	ค้นหาข้อมูลยาจากฐานข้อมูลกลาง	1. ช่อง Search ดึงข้อมูลจริงจาก <code>medications</code> ด้วย ILIKE query แบบ debounce (ไม่ยิง query ทุกตัวอักษร) 2. Loading Skeleton ระหว่างค้นหา, Empty State หากไม่พบยาที่ค้นหา
4.4	จำกัดสิทธิ์แก้ไขเฉพาะ Admin	1. เปิดใช้งาน RLS บน <code>medications</code> : SELECT เปิดให้ authenticated ทุกคน, INSERT/UPDATE/DELETE จำกัดเฉพาะ <code>role='staff'</code> (ตรวจสอบผ่าน subquery ไปยัง <code>profiles.role</code>) 2. Frontend ซ่อนปุ่ม Add/Edit/Delete จาก UI ของ <code>role='student'</code> เป็นการป้องกันชั้นที่สอง

ตาราง AC — ฟังก์ชันที่ 5: ระบบแจ้งเตือนการกินยาส่วนบุคคล

ID	User Story	Acceptance Criteria
5.1	เพิ่มรายการยาพร้อมกำหนดเวลาแจ้งเตือน	1. Form: ค้นหา/เลือกยาจาก <code>medications</code> (Autocomplete), กรอกขนาดที่ใช้จริง, จำนวนครั้งต่อวัน, เวลาแจ้งเตือน (Time Picker แบบเลือกได้หลายค่า), ช่วงวันที่เริ่ม-สิ้นสุด 2. Validation: ต้องเลือกยาและเวลาแจ้งเตือนอย่างน้อย 1 ช่วง ก่อนบันทึกได้ 3. บันทึก → INSERT ลง <code>medication_reminders</code> (<code>status='active'</code>) พร้อม Loading/Toast/Error State 4. RLS: INSERT อนุญาตเฉพาะเมื่อ <code>auth.uid() = user_id</code> ที่กำลังสร้าง
5.2	แก้ไข/หยุดรายการแจ้งเตือนยา	1. รายการยาของฉันทดึงข้อมูลจริงจาก <code>medication_reminders</code> WHERE <code>user_id = auth.uid()</code> พร้อมปุ่ม Edit / หยุดชั่วคราว (<code>status→paused</code>) / ลบ (Confirm Dialog) 2. RLS: UPDATE/DELETE อนุญาตเฉพาะ <code>auth.uid() = user_id</code> ของแถวนั้น; Loading/Error State ตามมาตรฐาน

5.3	บันทึกผล "กินแล้ว"/"ข้าม"	1. เมื่อถึงเวลาแจ้งเตือน (ตรวจสอบผ่าน Scheduled Function เทียบกับ reminder_times) แสดง Popup พร้อมปุ่ม "กินแล้ว"/"ข้าม" 2. กดปุ่ม → INSERT ลง medication_logs (status='taken'/'skipped', actual_datetime=now()) ทันที พร้อม Loading indicator สั้นๆ 3. RLS บน medication_logs: INSERT/SELECT อนุญาตเฉพาะเมื่อ reminder_id นั้นเป็นของผู้ใช้ปัจจุบัน
5.4	ดูสรุปประวัติการกินยาย้อนหลัง	1. ดึงข้อมูลจริงจาก medication_logs JOIN medication_reminders JOIN medications WHERE user_id = auth.uid() แสดงเป็นรายการ/ปฏิทิน พร้อมสรุป % ความสม่ำเสมอ 2. Loading Skeleton ขณะคำนวณ, Empty State หากยังไม่มีประวัติ
5.5	ข้อมูลยาส่วนตัวมองเห็นได้เฉพาะตนเอง	1. เปิดใช้งาน RLS บนทั้ง medication_reminders และ medication_logs โดยไม่มี Policy ใดที่อนุญาตให้ role='staff' หรือผู้อื่นอ่านข้อมูลได้ (Default Deny + Policy เฉพาะเจ้าของ) 2. Test Case: พยายาม query ข้าม user_id อื่นผ่าน Client SDK ต้องได้ผลลัพธ์ว่างเปล่า

ตาราง AC — ฟังก์ชันที่ 6: ศูนย์แจ้งเตือนและแดชบอร์ด

ID	User Story	Acceptance Criteria
6.1	เห็นรายการแจ้งเตือนทั้งหมดในที่เดียว	1. ไอคอนกระดิ่งที่ Header แสดง Badge จำนวน is_read=false ดึงข้อมูลจริงจาก notifications (Real-time หรือ Polling) 2. คลิกเปิดรายการที่ดึง notifications WHERE user_id=auth.uid() ORDER BY created_at DESC พร้อม Loading State
6.2	ทำเครื่องหมายอ่านแล้ว/ลบแจ้งเตือน	1. คลิกรายการ → UPDATE is_read=true และนำทางไปหน้าที่เกี่ยวข้อง (ตาม related_id) 2. ปุ่มลบต่อรายการมี Confirm Dialog ก่อนเรียก DELETE 3. RLS: SELECT/UPDATE/DELETE อนุญาตเฉพาะ auth.uid() = user_id

6.3	ดูแดชบอร์ดสรุปสถิติ	1. ดึงข้อมูลสรุปจริงจาก DB (COUNT นัดหมายวันนี้แยกตามสถานะ, กราฟ 7 วัน ย้อนหลัง, แผนกยอดนิยม) แสดงผ่าน Chart Component 2. Loading Skeleton ขณะคำนวณสถิติ, Error State หากล้มเหลว 3. Route Guard: เข้าถึงได้เฉพาะ role='staff'; ผู้ใช้ role='student' ที่พยายามเข้า URL ตรงต้องถูก redirect ออก
6.4	ส่งประกาศแจ้งเตือนไปยังผู้ใช้	1. Form ส่งประกาศ: หัวข้อ, ข้อความ, เลือกกลุ่มเป้าหมาย (ทั้งหมด /student/staff) 2. กดส่ง → INSERT ลง notifications แบบ Bulk สำหรับผู้ใช้ทุกคนในกลุ่มเป้าหมาย พร้อม Loading State 3. RLS: INSERT (แทนผู้ใช้อื่น) อนุญาตเฉพาะ role='staff'
6.5	ส่งแจ้งเตือนตรงเวลาคลาดเคลื่อนไม่เกิน 1 นาที	1. Scheduled Function (เช่น Supabase Edge Function + pg_cron) ทำงานทุก 1 นาที ตรวจสอบ medication_reminders และ appointments ที่ใกล้ถึงเวลา แล้วสร้าง Notification ทันที 2. Test Case: เวลาที่สร้างจริง (created_at) เทียบกับเวลาที่ควรแจ้งเตือนต้องคลาดเคลื่อนไม่เกิน 1 นาที

สรุปขั้นตอนถัดไป

เอกสารนี้ให้ข้อมูลครบสำหรับนำไปต่อยอดได้ทันที:

- **Use Case Diagram:** Actor 2 ตัว (นักศึกษา, เจ้าหน้าที่) และ Use Case สามารถอ้างอิงจาก User Stories ทั้ง 31 รายการ หรือสรุปย่อระดับพีเจอาร์ทั้ง 6 พีเจอาร์
- **ER Diagram:** ใช้ตารางทั้ง 9 ตารางและความสัมพันธ์ที่สรุปไว้ในหัวข้อ "ภาพรวมตารางฐานข้อมูล" เป็นจุดตั้งต้น
- **Product Backlog:** สามารถคัดลอกตาราง Acceptance Criteria ทั้ง 6 ตารางไปยัง Tool บริหารงาน (เช่น Trello, Jira, Notion) โดยแบ่ง Sprint ตามพีเจอาร์ที่สมาชิกแต่ละคนรับผิดชอบ